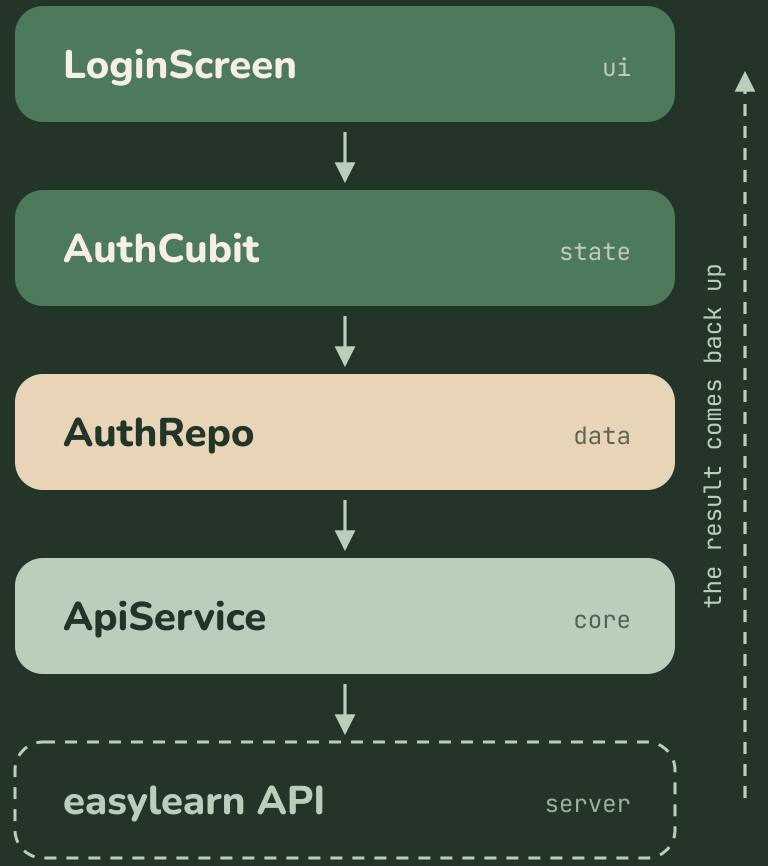
// flutter · clean architecture

Clean Architecture in Flutter

How the Easy Learn app goes from a button tap to the API and back.

هنفهم كل طبقة بتعمل إيه، وليه الكود متقسم كده، والداتا بتمشي إزاي من الشاشة للسيرفر وترجع.



Salma Elmaghawry
Software Engineer & Content Creator

// 01 · the app

Meet Easy Learn

01 Register
name, email, phone and password, plus an optional picture, address and location

02 Login
email and password, then the server sends a 6-digit code to your email

03 Verify code
checks the OTP and saves the token on the phone

04 Profile
shows your data, with logout in the app bar

05 Edit profile
update your data, picture and location

By the end of the session you can

- // Name the one job of every layer in the app
- // Trace a tap from the screen to the server and back
- // Explain how GetIt, routes and BlocProvider connect the pieces
- // Add a new feature with the same 7-step recipe
- // Add the Domain layer: entities, contracts and use cases
- // Give the app its own icon and get it ready for the store
- // Use AI to learn faster without losing the skill

التطبيق صغير، بس فيه كل اللي هتقابله في أي شغل حقيقي: API و token وصور ولوكيشن.

Everything in one widget

the wrong way · login_screen.dart

```dart
Future<void> _login() async {
  setState(() => _loading = true);
  try {
    final dio = Dio(BaseOptions(
      baseUrl: 'https://easylearn.devawy.com/api/',
      headers: {'x-api-key': 'wQ9KxY7...'},
    ));
    await dio.post('client/login', data: {
      'email': _email.text,
      'password': _password.text,
    });
    Navigator.pushNamed(context, 'otp', arguments: _email.text);
  } on DioException catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(e.response?.data['message'])),
    );
  }
  setState(() => _loading = false);
}
```

5 screens, 7 API calls

The base URL, API key and token code get copied into every one of them.

Change one header

You edit every screen instead of one file.

Can’t test it

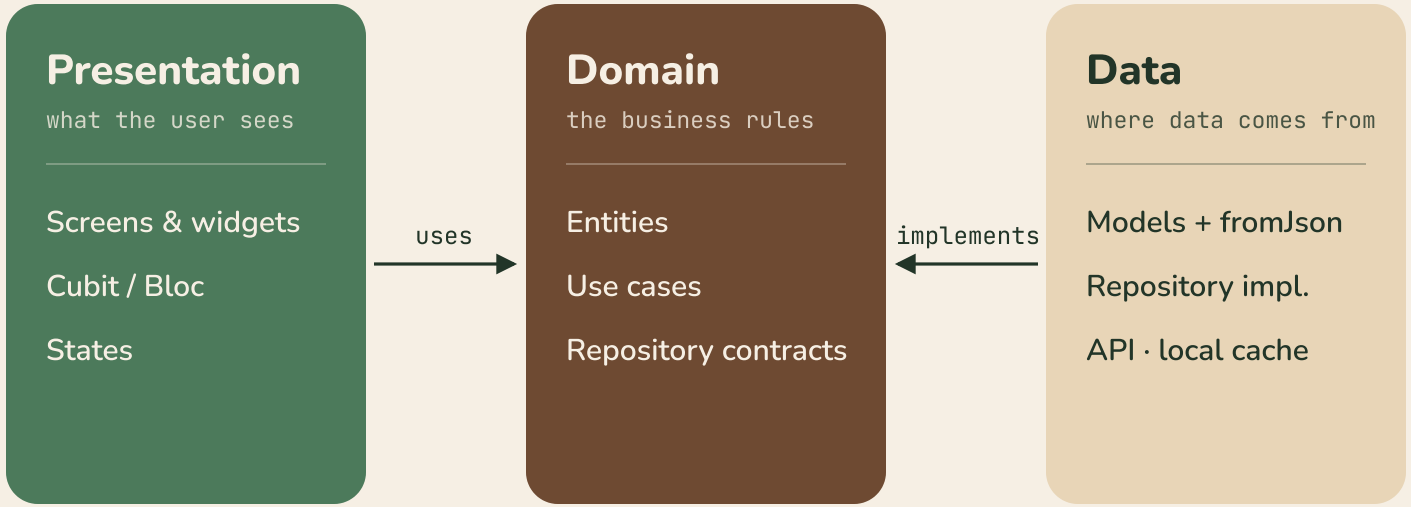
To test the login logic you have to run the whole screen.

One method, four jobs

Network, errors, navigation and loading state, all tangled together.

لو كل شاشة بتكلم السيرفر بنفسها، أي تغيير صغير هتلف عليه في كل الشاشات.

Split the app by responsibility



Dependency rule: every arrow points at Domain. Domain imports no Flutter and no Dio, so swapping the UI or the HTTP client doesn't touch your business rules.

Think of a restaurant

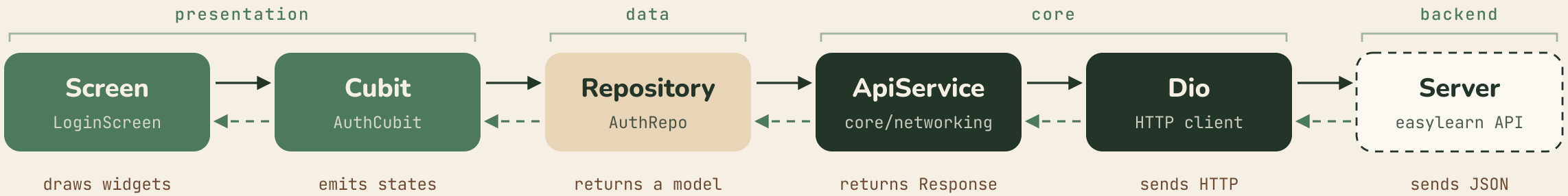
UI	**Waiter.** Takes your order, brings the plate.
Cubit	**Head chef.** Decides what happens next.
Repository	**Storekeeper.** Gets the ingredients.
API	**Supplier.** Owns the raw food.

The waiter never calls the supplier.

زي المطعم بالظبط: الجرسون عمره ما بيروح للمورد بنفسه. كل واحد عارف شغلته.

// 04 · in this project

Our version, kept simple



What we skipped, for now: Domain

// The Cubit calls the Repository directly

// The Repository returns models like `ProfileModel`, not entities

// No use cases, no abstract repository

// **We add all of it in part 3**

Add Domain when...

// Several cubits share the same business rule

// Data comes from 2 sources, like the API and a local DB

// You want to test the logic with a fake repository

إحنا شيلنا الـ Domain عشان التطبيق لسه صغير. لما المنطق يكبر، بنضيفها في النص بين الـ Cubit والـ Repo.

Feature-first folders

```
lib/
├── main.dart              reads the token, picks the first screen
├── core/                  shared by every feature
│   ├── cache/             CacheHelper · secure token
│   ├── di/                GetIt setup
│   ├── helpers/           location · image picker · phone
│   ├── networking/        ApiService · ApiConstants · ApiErrorHandler
│   ├── routes/            Routes · AppRouter
│   └── widgets/           AppButton · PicturePicker · LocationButton
└── feature/
    ├── auth/
    │   ├── data/          AuthRepo · RegisterRequestBody
    │   └── presentation/
    │       ├── cubit/     AuthCubit · AuthState
    │       └── ui/        Login · Register · Otp screens
    └── profile/
        ├── cubit/         ProfileCubit · ProfileState
        ├── data/          ProfileRepo · ProfileModel · request body
        └── ui/            Profile · EditProfile screens
```

`core/`

Code any feature can use: networking, cache, helpers, shared widgets. **di/** and **routes/** are the wiring, the only places that know every feature.

`feature/`

One folder per feature. Inside it, **data/** talks to the API, **cubit/** holds the state, **ui/** draws.

Spot the difference: auth/ puts cubit and ui inside presentation/, profile/ keeps them side by side. Both work. Pick one style and use it everywhere.

كل feature ليها فولدر لوحدها. لو حد سألك الـ login فين، هتروح عليه على طول.

// 06 · who does what

One job per layer

Layer	In our app	Its one job	It must NOT
Screen	LoginScreen ProfileScreen	Read input, call the cubit, draw each state, navigate and show SnackBars	Call Dio, parse JSON, decide business rules
Cubit	AuthCubit ProfileCubit	Call the repo, turn the result into states	Import widgets or BuildContext, know URLs
Repository	AuthRepo ProfileRepo	Call the right endpoint, turn JSON into models, save or delete the token	Emit states or show messages
Model / body	ProfileModel RegisterRequestBody	Describe the data: `fromJson` in, `toFormData` out	Call the API
Core	ApiService ApiErrorHandler · CacheHelper	Base URL, headers, token interceptor, error to message, secure storage	Know which screen is calling

لو لقيت طبقة بتعمل شغل طبقة تانية، يبقى في حاجة غلط في التقسيم.

One door to the server

core/networking/api_service.dart

```dart
ApiService(this.dio) {
  dio.options.baseUrl = ApiConstants.apiBaseUrl;
  dio.options.connectTimeout = const Duration(seconds: 20);
  dio.options.receiveTimeout = const Duration(seconds: 20);
  dio.options.headers = {
    'Accept': 'application/json',
    'x-api-key': ApiConstants.apiKey,
  };
  dio.interceptors.add(InterceptorsWrapper(onRequest: _addToken));
  if (kDebugMode) dio.interceptors.add(PrettyDioLogger(...));
}

Future<void> _addToken(options, handler) async {
  final token = await CacheHelper.getSecureData(
    key: CacheConstants.userToken,
  );
  if (token != null) {
    options.headers['Authorization'] = 'Bearer $token';
  }
  handler.next(options);
}
```

`AuthRepo` calls `post('client/login', data)`

↓ `baseUrl` — `https://easylearn.devawy.com/api/` + the endpoint

↓ `headers` — `Accept` and `x-api-key` on every request

↓ `_addToken` — `Authorization: Bearer <token>`, if one is saved

↓ `logger` — PrettyDioLogger prints it, in debug builds only

↓ `server` — the request leaves the app

أي request في التطبيق بيعدي من الباب ده، فالـ token والـ headers بيتكتبوا مرة واحدة بس.

Models translate JSON and Dart

GET client/profile coming in

`response.data['data']`

```
{
  "name": "Ahmed",
  "phone": "+201000000000",
  "latitude": "30.0444",
  "picture_url": "https://…"
}
```

`ProfileModel.fromJson(json)` →

Dart object

```
ProfileModel(
  name: 'Ahmed',
  phone: '+201000000000',
  latitude: 30.0444,   // double
  pictureUrl: 'https://…',
)
```

POST client/register going out

Dart object

```
RegisterRequestBody(
  name: 'Ahmed',
  email: 'ahmed@mail.com',
  password: '••••••••',
  picturePath: null,
)
```

`await body.toFormData()` →

FormData (multipart)

```
name: Ahmed
email: ahmed@mail.com
password: ••••••••
password_confirmation: ••••••••
// no picture key, it was null
```

Strings to numbers

Laravel sends decimal columns as strings, so `fromJson` uses `double.tryParse`.

Skip empty fields

`if (address != null) 'address': address` keeps nulls out of the request.

POST that acts as PUT

Update sends `_method: PUT`: PHP only reads multipart bodies (our picture) on POST.

الموديل ده مترجم: بيحوّل كلام السيرفر (JSON) لكلام Dart، والعكس.

The repository fetches, nothing more

feature/auth/data/auth_repo.dart

```dart
class AuthRepo {
  final ApiService _apiService;
  AuthRepo(this._apiService);
  Future<String> login({
    required String email,
    required String password,
  }) async {
    final response = await _apiService.post(
      ApiConstants.login,
      data: {'email': email, 'password': password},
    );
    return response.data['message'];
  }
  Future<String> verifyOtp({...}) async {
    final response = await _apiService.post(ApiConstants.verifyOtp, ...);
    await CacheHelper.saveSecureData(
      key: CacheConstants.userToken, value: response.data['token'],
    );
    return response.data['message'];
  }
}
```

Gets ApiService from its constructor. GetIt passes it in, the repo never creates it.

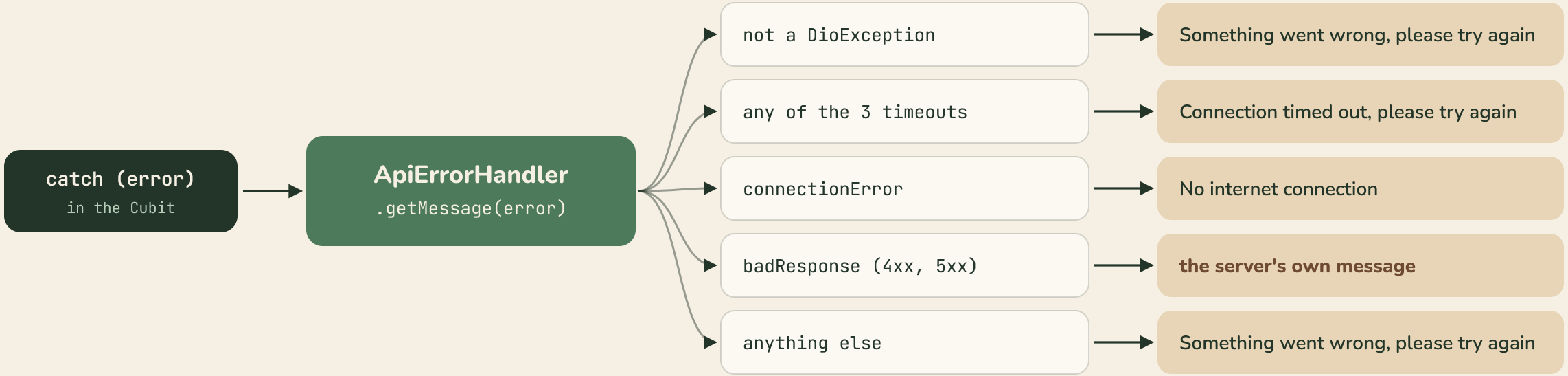
Returns plain values: `Future<String>`, `Future<ProfileModel>`.

Doesn't catch errors. If the request fails, Dio throws and the error goes up to the cubit.

Data side effects live here: save the token after `verifyOtp`, delete it in `logout`'s `finally`.

الـ Repo مايعرفش مين طلب الداتا ولا هتتعرض إزاي.
بيجيبها وبس.

Turn any error into a message



badResponse, 2 shapes: `"message": "..."` is shown as is.
`"message": {"email": ["..."], ...}` becomes one line per error.

Then: `emit(AuthFailure(message))` and the listener shows it in a SnackBar.

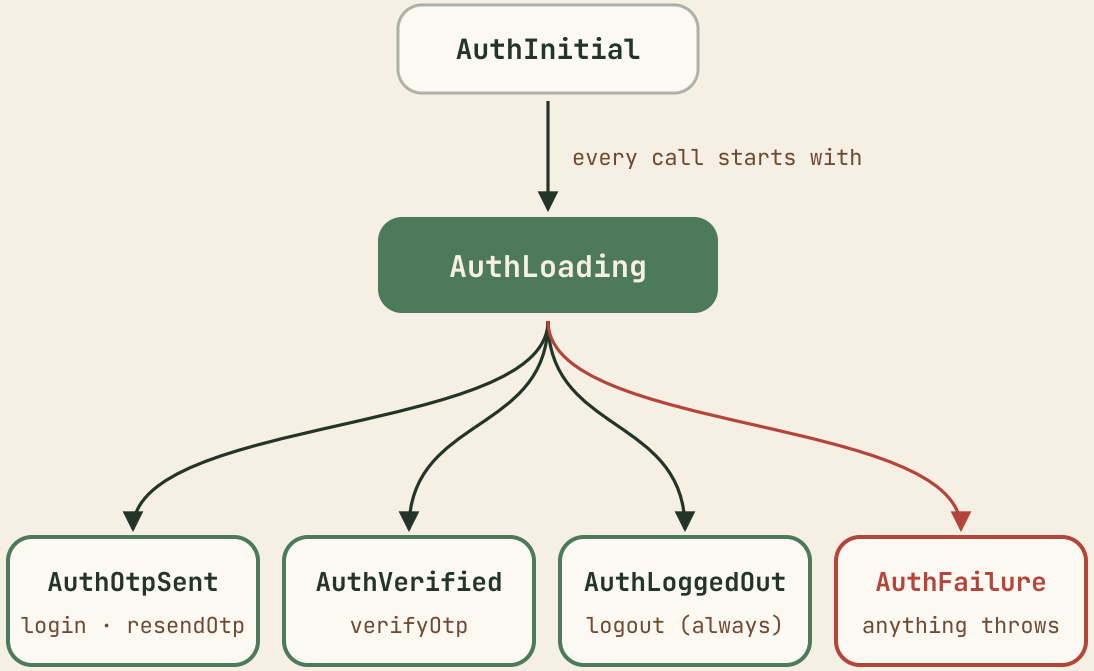
اليوزر عمره ما يشوف DioException. بيشوف جملة مفهومة يقدر يتصرف على أساسها.

The cubit turns results into states

feature/auth/presentation/cubit/auth_cubit.dart

```dart
class AuthCubit extends Cubit<AuthState> {
  final AuthRepo _authRepo;
  AuthCubit(this._authRepo) : super(AuthInitial());

  Future<void> login({
    required String email,
    required String password,
  }) async {
    emit(AuthLoading());
    try {
      final message = await _authRepo.login(
        email: email, password: password,
      );
      emit(AuthOtpSent(email: email, message: message));
    } catch (error) {
      emit(AuthFailure(ApiErrorHandler.getMessage(error)));
    }
  }
}
```



`sealed class AuthState` : the compiler knows every state the screen can receive.

الـ Cubit بيقول للشاشة الحكاية وصلت لفين: لسه بيحمّل، نجح، ولا فشل.

listener acts, builder draws

feature/auth/presentation/ui/login_screen.dart

```dart
BlocConsumer<AuthCubit, AuthState>(
  listener: (context, state) {
    if (state is AuthOtpSent) {
      Navigator.pushNamed(context, Routes.otp, arguments: state.email);
    } else if (state is AuthFailure) {
      ScaffoldMessenger.of(context)
          .showSnackBar(SnackBar(content: Text(state.message)));
    }
  },
  builder: (context, state) {
    return ListView(children: [
      // email + password fields ...
      AppButton(
        text: 'Login',
        isLoading: state is AuthLoading,
        onPressed: _login,
      ),
    ]);
  },
)
```

`listener` · runs once per new state

Side effects: navigation, SnackBars, dialogs.

`builder` · can run many times

Returns widgets only. The spinner comes from `isLoading: state is AuthLoading`.

The trigger

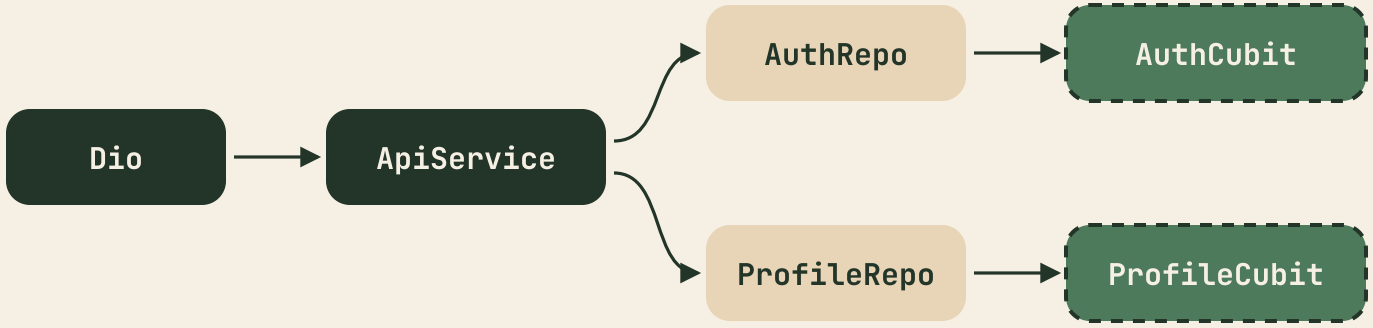
`_login()` only calls `context.read<AuthCubit>().login(...)`.

Navigator.push inside builder can open the same screen twice, because builder may run again for the same state.

اللي بيحصل مرة واحدة زي التنقل أو الرسالة يبقى في الـ listener. اللي بيترسم يبقى في الـ builder.

GetIt builds the chain for us

A → B: A is passed into B's constructor



core/di/dependency_injection.dart

```dart
void setupGetIt() {
  getIt.registerLazySingleton<Dio>(() => Dio());
  getIt.registerLazySingleton<ApiService>(() => ApiService(getIt()));

  getIt.registerLazySingleton<AuthRepo>(() => AuthRepo(getIt()));
  getIt.registerFactory<AuthCubit>(() => AuthCubit(getIt()));

  getIt.registerLazySingleton<ProfileRepo>(() => ProfileRepo(getIt()));
  getIt.registerFactory<ProfileCubit>(() => ProfileCubit(getIt()));
}
```

lazySingleton: one shared object, created the first time someone asks.

factory: a brand new object every time.

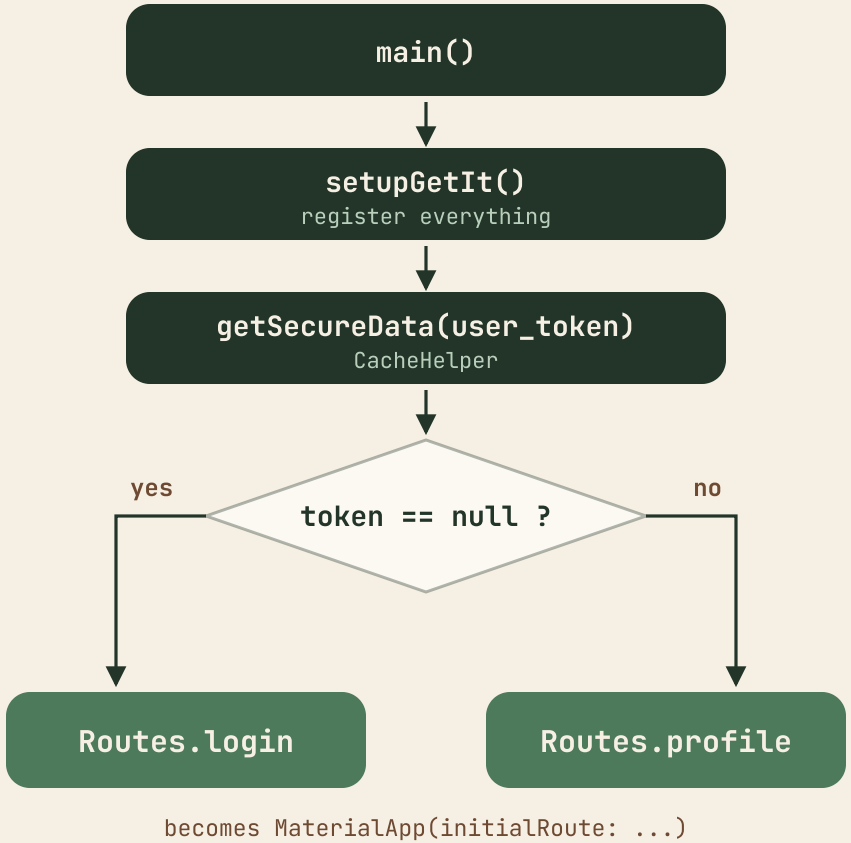
Why factory for cubits?

BlocProvider closes the cubit when its screen goes away. A closed cubit can't emit again, so every screen asks GetIt for a fresh one.

`getIt()` inside a constructor means: give me whatever type this parameter needs.

بدل ما كل شاشة تعمل Dio و ApiService و Repo بإيدها، بنعرّف GetIt مرة واحدة وهو يجهّز الباقي.

Each route brings its own cubit



becomes `MaterialApp(initialRoute: ...)`

Route name	Screen	BlocProvider creates	arguments
`login + '/'`	LoginScreen	`AuthCubit`	`none`
`register`	RegisterScreen	`AuthCubit`	`none`
`otp`	OtpScreen	`AuthCubit`	`email (String)`
`profile`	ProfileScreen	`ProfileCubit..getProfile()` + `AuthCubit`	`none`
`editProfile`	EditProfileScreen	`ProfileCubit`	`ProfileModel`

An unknown name makes `generateRoute` return `null`, so `onUnknownRoute` shows “Page not found”.

الـ route هو اللي بيجهّز الـ Cubit، والشاشة بتستخدمه على طول من غير ما تعرف جه منين.

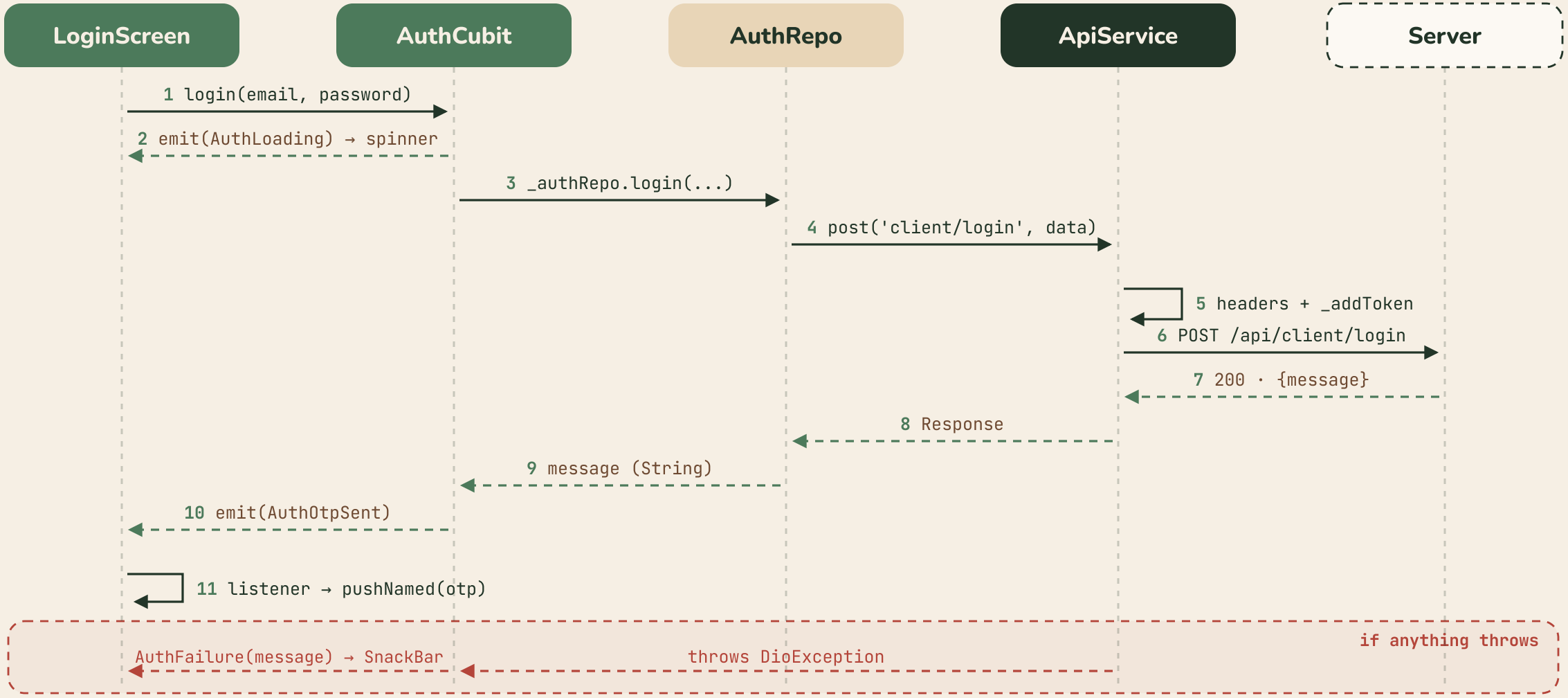
// part 2 · the flow

Follow one tap, end to end

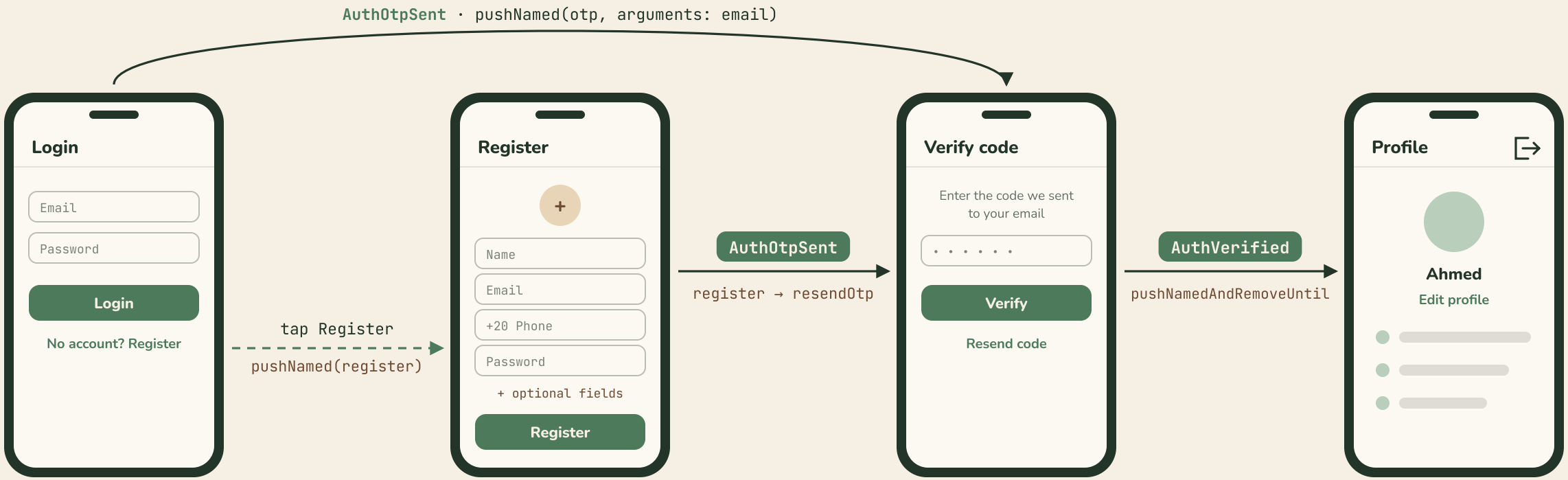
Three journeys through the real code: login, sign up and profile.

هنمشي ورا الداتا خطوة بخطوة، من أول ما تدوس الزرار لحد ما الشاشة تتغير.

Tap Login: who talks to whom



From sign up to profile



الـ register مابيبعتش كود لوحده، فالـ Cubit بيطلب resendOtp على طول عشان اليوزر يوصله الكود.

Profile: load, edit, logout

A Open profile

route creates the cubit

getIt<ProfileCubit>()..getProfile() → ProfileLoading · spinner → GET client/profile → ProfileModel.fromJson → ProfileSuccess(profile) → _ProfileDetails draws it

B Edit profile

its own fresh cubit

tap Edit profile → pushNamed(editProfile, arguments: profile) → updateProfile(body) → POST client/profile/update · _method: PUT → ProfileUpdated → SnackBar + pop(true) → updated == true → getProfile()

C Logout

uses AuthCubit

tap logout icon → AuthCubit.logout() → POST client/logout → finally: delete token → AuthLoggedOut, even if the call failed → pushNamedAndRemoveUntil(login)

screen action | cubit · state | data | core · API

صفحة التعديل ليها Cubit جديد. ولما ترجع بـ true، صفحة البروفايل بتعمل getProfile تاني عشان تجيب الداتا الجديدة.

The life of a token

1 — 2 — 3 — 4

// save

After verifyOtp

`AuthRepo`

```
saveSecureData(
  key: user_token,
  value: token,
)
```

// read

When the app starts

`main()`

```
token == null
  ? Routes.login
  : Routes.profile
```

// send

On every request

`ApiService`

```
headers['Authorization']
  = 'Bearer $token'
```

// delete

On logout

`AuthRepo`

```
finally {
  deleteSecureData(
    user_token)
}
```

Where it lives: `flutter_secure_storage`, encrypted with the Keychain on iOS and the Keystore on Android. SharedPreferences would keep it as plain text.

الـ token بيتحفظ مرة، ويتقري مع كل request، ويتمسح أول ما تعمل logout.

A new feature in 7 steps

Example: an Orders feature. The endpoint name is made up, use the one your backend gives you.

01 Endpoint

`core/networking/api_constants.dart`

Add `static const String orders = "client/orders";`

02 Model / request body

`feature/orders/data/`

`fromJson` for what comes in, `toFormData` or a map for what goes out

03 Repository

`feature/orders/data/orders_repo.dart`

One method per endpoint. Returns plain values, lets errors throw

04 States + Cubit

`feature/orders/cubit/`

Loading, Success, Failure. `try/catch` with `ApiErrorHandler`

05 GetIt

`core/di/dependency_injection.dart`

Repo: `registerLazySingleton`. Cubit: `registerFactory`

06 Route

`core/routes/`

A name in `Routes`, a case in `AppRouter` wrapped in `BlocProvider`

07 Screen

`feature/orders/ui/`

`BlocConsumer`: listener acts, builder draws

Five classic mistakes

~~Calling Dio inside a widget~~	→	Screen → Cubit → Repo → ApiService
~~Navigator.push inside builder~~	→	Navigate in the listener, it runs once per state
~~registerLazySingleton for a Cubit~~	→	registerFactory: a closed cubit can’t emit again
~~Showing error.toString() to the user~~	→	ApiErrorHandler.getMessage(error)
~~Adding the token in every repo~~	→	The _addToken interceptor does it once

الغلطات دي بتشتغل عادي في الأول، وبتوجعك بعدين لما التطبيق يكبر.

// part 3 · level up

Add the Domain layer

Entities, repository contracts and use cases, added to the same app. The project you take home has this version.

لحد هنا كنا شايلين الـ Domain عشان نبسّطها. دلوقتي هنضيفها ونشوف إيه اللي بيتغير.

Three new pieces in domain/

entity

Entity

The data as your app means it. Plain Dart: no JSON, no Dio, no Flutter.

`ProfileEntity` `RegisterEntity` `UpdateProfileEntity`

contract

Repository contract

An abstract class: **what** data the app needs, not **how** to get it. data/ implements it.

`AuthRepo` `ProfileRepo`

use case

Use case

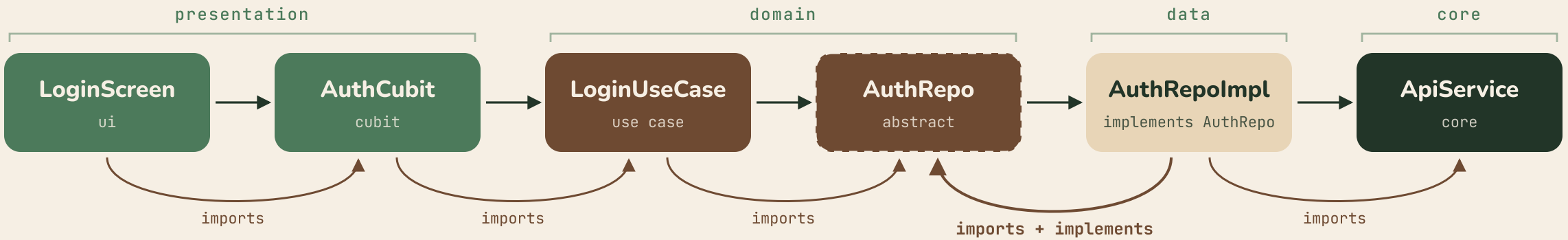
One thing the user can do, in one class with `call()`. Business rules live here.

`LoginUseCase` `RegisterUseCase` `GetProfileUseCase` `+ 4 more`

The rule: `domain/` imports only `domain/`. Presentation and data both import it, never the other way round.

الـ Domain هي قلب التطبيق: بتقول التطبيق بيعمل إيه، من غير ما تعرف الداتا جاية منين ولا هتتعرض إزاي.

Calls go right, imports point in



Follow the calls: they go right, all the way to `AuthRepoImpl` and the server.

Follow the imports: `data/` points back at `domain/`. Domain never imports data. That's the rule from slide 4.

Who joins them? GetIt. Anyone who asks for `AuthRepo` gets an `AuthRepoImpl`.

الـ Cubit بيكلم AuthRepo ومش عارف إن اللي بيرد عليه AuthRepoImpl. ده اللي بيخلينا نغيّر الـ data من غير ما نلمس الباقي.

Entity vs Model

domain/entities/profile_entity.dart

```dart
class ProfileEntity {
  final String name;
  final String email;
  final double? latitude;
  // + phone, address, longitude, pictureUrl
  ProfileEntity({required this.name, ...});
}
```

data/profile_model.dart

```dart
class ProfileModel extends ProfileEntity {
  ProfileModel({required super.name, ...});
  factory ProfileModel.fromJson(Map<String, dynamic> json) {
    return ProfileModel(
      name: json['name'],
      latitude: double.tryParse('${json['latitude']}'),
      ...
    );
  }
}
```

Coming in

The **Model extends the Entity** and adds `fromJson`. The repo returns it typed as `ProfileEntity`, so the cubit and screens never see JSON.

data/register_request_body.dart

```dart
extension RegisterRequestBody on RegisterEntity {
  Future<FormData> toFormData() async {
    return FormData.fromMap({'name': name, ...});
  }
}
```

Going out

An **extension** adds `toFormData` to the entity inside `data/`, so domain never imports Dio.

الـ Entity هي الداتا زي ما التطبيق فاهمها. الـ Model هي نفس الداتا زي ما السيرفر بيبعتها.

The contract and its implementation

domain/repositories/auth_repo.dart

```dart
abstract class AuthRepo {
  Future<String> login({required String email, required String password});
  Future<String> register(RegisterEntity entity);
  Future<String> verifyOtp({required String email, required String otpCode});
  Future<String> resendOtp({required String email});
  Future<String> logout();
}
```

abstract class = a promise. Every method, no body.

@override: the impl must keep every promise, or the app won't compile.

Why bother? A test can pass a fake `AuthRepo`, or you can add a local cache, and the cubit and use cases don't change.

data/auth_repo_impl.dart · was auth_repo.dart

```dart
class AuthRepoImpl implements AuthRepo {
  final ApiService _apiService;

  AuthRepoImpl(this._apiService);

  @override
  Future<String> register(RegisterEntity entity) async {
    final response = await _apiService.post(
      ApiConstants.register,
      data: await entity.toFormData(),
    );
    return response.data['message'];
  }

  // login, verifyOtp, resendOtp, logout: same code as before
}
```

الـ abstract class زي العقد: بيقول هنجيب إيه. الـ Impl هو اللي بيقول هنجيبه إزاي.

Use cases: one action, one class

thin most of ours just pass the call through

domain/usecases/login_usecase.dart

```dart
class LoginUseCase {
  final AuthRepo _authRepo;

  LoginUseCase(this._authRepo);

  Future<String> call({required String email, required String password}) {
    return _authRepo.login(email: email, password: password);
  }
}
```

`call()` lets you use the object like a function:
`await _loginUseCase(email: e, password: p)`

a real rule moved here from the cubit

domain/usecases/register_usecase.dart

```dart
class RegisterUseCase {
  final AuthRepo _authRepo;

  RegisterUseCase(this._authRepo);

  // the backend doesn't send an OTP on register,
  // so we ask for one right away
  Future<String> call(RegisterEntity entity) async {
    await _authRepo.register(entity);
    return _authRepo.resendOtp(email: entity.email);
  }
}
```

The cubit used to call register, then resendOtp. Now that rule has one home, and any screen that registers gets it for free.

الـ use case الرفيع ممكن يبان مالوش لازمة، بس هو المكان الطبيعي لأي rule هتيجي بعدين، زي register وبعدها resendOtp.

The cubit and GetIt, after

presentation/cubit/auth_cubit.dart

```dart
class AuthCubit extends Cubit<AuthState> {
  final LoginUseCase _loginUseCase;
  final RegisterUseCase _registerUseCase;
  // + VerifyOtp, ResendOtp and Logout use cases

  Future<void> register(RegisterEntity entity) async {
    emit(AuthLoading());
    try {
      final message = await _registerUseCase(entity);
      emit(AuthOtpSent(email: entity.email, message: message));
    } catch (error) {
      emit(AuthFailure(ApiErrorHandler.getMessage(error)));
    }
  }
}
```

core/di/dependency_injection.dart

```dart
getIt.registerLazySingleton<AuthRepo>(() => AuthRepoImpl(getIt()));
getIt.registerLazySingleton<LoginUseCase>(() => LoginUseCase(getIt()));
// + one line per use case
getIt.registerFactory<AuthCubit>(
  () => AuthCubit(getIt(), getIt(), getIt(), getIt(), getIt()),
);
```

Ask for the contract, get the implementation: `<AuthRepo>` is the type, `AuthRepoImpl` is what GetIt builds.

Use cases hold no state, so one shared instance is enough. Cubits stay `factory`.

Register is calmer now: before, the cubit emitted Loading twice. Now it's Loading, then OtpSent.

الـ Cubit بقى بيكلم use cases بس. مابقاش يعرف حاجة عن الـ repo ولا الـ API.

The folders now, and is it worth it?

```
feature/auth/
├── domain/                          new · pure Dart
│   ├── entities/                    RegisterEntity
│   ├── repositories/                AuthRepo (abstract)
│   └── usecases/                    Login · Register · VerifyOtp · ResendOtp · Logout
├── data/
│   ├── auth_repo_impl.dart          was auth_repo.dart
│   └── register_request_body.dart   now an extension
└── presentation/                    talks to domain only

feature/profile/
├── domain/                          new · 2 entities, ProfileRepo, 2 use cases
├── data/                            ProfileModel extends ProfileEntity, ProfileRepoImpl
└── cubit/ · ui/                     use ProfileEntity, not ProfileModel
```

What it costs

12 new files in `domain/` for the same two features.

What you get

Change the data source without touching cubits, test with fakes, and give every business rule one home.

The recipe grows: the 7 steps get 3 more, an entity, a repository contract and a use case.

مش لازم كل تطبيق صغير يبقى فيه Domain. بس أول ما التطبيق يكبر أو تشتغلوا في تيم، هتفرق معاكم جداً.

// part 4 · ship it

From your laptop to the store

The app icon, Google Play and the App Store, and how to use AI without losing the skill.

دي آخر سيشن في الكورس، فهنتكلم عن اللي بعد الكود: إزاي التطبيق بتاعك يوصل لإيد الناس.

Give the app its own icon

01 **Add the package**
dev only, it never ships inside your app

```
flutter pub add dev:flutter_launcher_icons
```

02 **Add one 1024×1024 PNG**
a full square in `assets/icon/`, no transparency for iOS

03 **Tell it what to make**
a `flutter_launcher_icons:` block in `pubspec.yaml`

04 **Run it**
it writes every size for Android and iOS

```
dart run flutter_launcher_icons
```

05 **Rebuild the app**
hot reload never changes the icon. Stop and run again, uninstall if the old one sticks



Flutter’s default icon, then ours: masked by iOS (rounded square) and by an Android launcher (circle).

الأيقونة أول حاجة اليوزر بيشوفها قبل ما يفتح التطبيق، فماينفعش نسيب لوجو Flutter.

What one command generated

pubspec.yaml

```yaml
dev_dependencies:
  flutter_launcher_icons: ^0.14.4

flutter_launcher_icons:
  android: true
  ios: true
  image_path: "assets/icon/icon.png"
  remove_alpha_ios: true
  adaptive_icon_background: "#673AB7"
  adaptive_icon_foreground: "assets/icon/icon_foreground.png"
  adaptive_icon_foreground_inset: 0
```



background foreground adaptive icon

what one command generated

```
android/app/src/main/res/
├── mipmap-mdpi … xxxhdpi/    ic_launcher.png, 48 → 192 px
├── drawable-*/               ic_launcher_foreground.png
├── mipmap-anydpi-v26/        ic_launcher.xml (adaptive)
└── values/colors.xml         the #673AB7 background
ios/Runner/Assets.xcassets/
└── AppIcon.appiconset/       21 PNGs, 20 → 1024 px
```

Adaptive icons: Android stacks the two layers and cuts them into a circle, squircle or square depending on the phone. Keep the drawing inside the middle so nothing gets cut.

Read your git diff

In our project the tool also changed an Xcode setting that only takes YES/NO to `AppIcon`. We put it back. Any tool that edits your project can be wrong.

Before you upload: audit this project

done	**App icon**	Done today with flutter_launcher_icons.
done	**Permission reasons**	`Info.plist` explains why we need the camera, photos and location.
to do	**Package name**	`com.example.clean_arch_session` : Play rejects `com.example`, and the name is locked after the first upload. Pick something like `com.easylearn.app`.
to do	**App name**	Android shows `clean_arch_session`, iOS shows `Clean Arch Session`. Make both **Easy Learn**.
to do	**Release signing**	Release builds still use the debug key. Create an upload key (next slide).
to do	**Account deletion**	Users can register, so both stores require a way to delete the account, in the app and on the web.
to do	**Privacy policy**	A public URL saying what we collect: name, email, phone, photo, address and location.
every upload	**Version**	`version: 1.0.0+1` : the number after `+` must go up with every upload.

المشروع بتاعنا شغال، بس لسه مش جاهز للستور. دي الحاجات اللي هتعملوها في أي تطبيق قبل ما ترفعوه.

Sign it, then build the bundle

1 create the upload key, once

```
keytool -genkey -v -keystore ~/upload-keystore.jks \
  -keyalg RSA -keysize 2048 -validity 10000 \
  -alias upload
```

2 android/key.properties, never committed

```
storePassword=••••••••
keyPassword=••••••••
keyAlias=upload
storeFile=/Users/you/upload-keystore.jks
```

4 build the bundle Play wants

```
flutter build appbundle
// → build/app/outputs/bundle/release/app-release.aab
```

Back up the .jks and its passwords outside the repo. Flutter's `android/.gitignore` already ignores `key.properties` and `*.jks`.

3 android/app/build.gradle.kts

```
import java.io.FileInputStream
import java.util.Properties

val keystoreProperties = Properties()
val keystoreFile = rootProject.file("key.properties")
if (keystoreFile.exists()) {
    keystoreProperties.load(FileInputStream(keystoreFile))
}

android {
    signingConfigs {
        create("release") {
            keyAlias = keystoreProperties["keyAlias"] as String
            keyPassword = keystoreProperties["keyPassword"] as String
            storeFile = keystoreProperties["storeFile"]?.let { file(it) }
            storePassword = keystoreProperties["storePassword"] as String
        }
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
        }
    }
}
```

Google Play, step by step

01 **Developer account**

$25, paid once. Use an email you'll keep for years.

02 **Create the app**

Name, default language, free or paid. The package name is locked after the first upload.

03 **Store listing**

Title (30 chars), short description (80), full description (4000), 512×512 icon, 1024×500 feature graphic, phone screenshots.

04 **App content**

Privacy policy, data safety, content rating, target audience. **App access:** our app needs a login, so give the reviewers a working test account.

05 **Test, then production**

New personal accounts run a **closed test with 12+ testers for 14 days**, then apply for production and promote the release.

الـ 14 يوم بتوع الـ closed testing بيتنسوا كتير. ابدأوا تجمعوا الـ testers من بدري، ومتغيروش الـ test account وقت المراجعة.

And the App Store?

	Google Play	App Store
Account	$25, once	$99 every year
You need	Any computer	A Mac with Xcode
Build	`flutter build appbundle` → .aab	`flutter build ipa` → .ipa
Upload	Play Console	Xcode Organizer or the Transporter app
Testing	Internal and closed testing tracks	TestFlight
App ID	`applicationId` in build.gradle.kts	Bundle ID in Xcode, same rules: no `com.example`

Apple reviews by hand

// Give them a test account, like on Play

// Account deletion must be inside the app

// Every permission needs a clear reason. Ours are in `Info.plist`

// Fill in the App Privacy answers in App Store Connect

نفس الفكرة بالظبط، بس أبل بتراجع بإيدها، فأي حاجة ناقصة بترجعلك برفض فيه السبب.

A pair programmer, not a replacement

Use it to

- // **Explain an error.** Paste the full error, the code, and what you expected.
- // **Explain code.** “Walk me through AuthCubit line by line.”
- // **Write boilerplate you understand:** fromJson, request bodies, states.
- // **Review your code.** “What breaks if the token is null?”
- // **Write tests** and find edge cases you missed.

Don’t

- // Paste code you can’t explain into your project.
- // Share secrets: API keys, keystore passwords, user tokens. Our `ApiConstants.apiKey` is one.
- // Trust a package name or version before checking pub.dev.
- // Let it choose your architecture. Tell it yours.
- // Skip the struggle while learning. Try first, then ask.

// chat ChatGPT · Claude · Gemini

// in your editor GitHub Copilot · Cursor · Claude Code · Gemini in Android Studio

الـ AI بيسرّعك لو انت فاهم، وبيوقعك لو بتنسخ وخلاص.

A prompt that gets good code

Context	your stack, your pattern, the real files
Task	one clear feature, with the endpoint
Rules	what it must follow and must not add
Output	what you want back, and ask it to explain

ask → read every line → run + analyze → explain it → commit

example prompt · the endpoint is made up

```
Context: Flutter app, feature-first Clean Architecture.
Screen → Cubit → UseCase → AuthRepo (domain contract)
→ AuthRepoImpl (data) → ApiService (Dio). GetIt for DI.
Here are LoginUseCase, AuthRepo, AuthRepoImpl and AuthCubit:
<paste them>

Task: add "Delete account". Call DELETE client/account,
then send the user back to Login.

Rules: follow the same pattern: one use case, a method on
the contract and the impl, try/catch with ApiErrorHandler
in the cubit. Domain imports nothing from data.
No new packages.

Output: only the changed files, then explain each change.
```

If you can’t explain a line, it doesn’t go in your project.

كل ما تديله context أكتر عن مشروعك، الكود اللي بيطلّعه بيبقى شبه كودك.

// end of the course

You built a real app. Now ship it.

Thank you for every question, every bug, and every “it works!”

شكراً على كل سؤال وكل bug وكل «اشتغل!». مستنية أشوف تطبيقاتكم على الستور.

Salma Elmaghawry
Software Engineer & Content Creator